

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 01. Patrones de diseño creacionales

Tema:

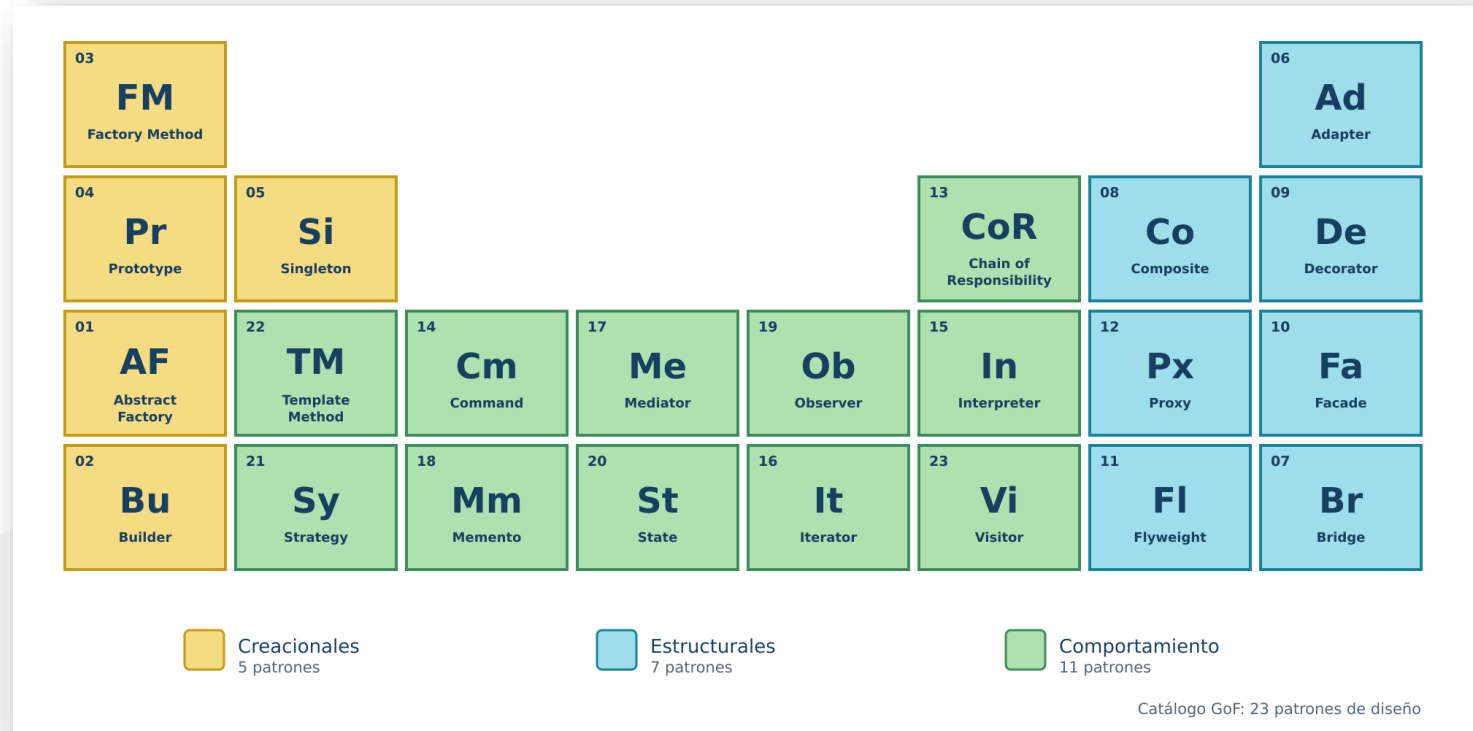
Introducción a los patrones creacionales

- *Introducción a los patrones creacionales*
- Factory Method
- Builder
- Singleton

Introducción a los patrones creacionales

Clasificación de los patrones GoF por propósito

- **Patrones creacionales (5):** Abstraen el proceso de creación de objetos y reducen la dependencia respecto de clases concretas.
- **Patrones estructurales (7):** Definen cómo combinar clases y objetos para construir estructuras mayores, flexibles y mantenibles.
- **Patrones de comportamiento (11):** Definen cómo los objetos se comunican, colaboran y distribuyen responsabilidades para realizar una tarea.



Introducción a los patrones creacionales

Introducción

- Los patrones creacionales abstraen y controlan el proceso de creación de objetos.
- Permiten decidir qué objeto crear, cómo configurarlo y cómo construirlo sin acoplar innecesariamente al cliente con clases concretas.
- Facilitan que el sistema sea independiente de cómo se crean, componen o representan determinados objetos.
- No eliminan el uso de new; encapsulan o delegan las decisiones de creación donde resulte conveniente.
- Los patrones creacionales pueden actuar principalmente mediante:
 - Herencia: una subclase decide qué clase concreta se instancia.
 - Delegación: otro objeto recibe la responsabilidad de crear o construir las instancias.
- Su aplicación debe responder a un problema real de diseño; utilizarlos sin necesidad puede aumentar innecesariamente la complejidad.

Introducción a los patrones creacionales

Catálogo

- **Factory Method:** Define un método para crear objetos, permitiendo que las subclases decidan qué clase concreta instanciar.
- **Builder:** Separa la construcción de un objeto complejo de su representación, permitiendo utilizar un mismo proceso de construcción para obtener diferentes representaciones.
- **Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
- **Prototype:** Permite crear nuevos objetos mediante la copia de una instancia existente utilizada como prototipo.
- **Singleton:** Garantiza que una clase tenga una única instancia y proporciona un punto de acceso controlado a ella.



Introducción a los patrones creacionales

¿Cómo entender un patrón de diseño?

Para analizar un patrón, debemos responder cinco preguntas:

- ¿Qué problema recurrente resuelve?
- ¿En qué contexto resulta aplicable?
- ¿Qué participantes intervienen y cómo colaboran?
- ¿Qué estructura propone?
- ¿Qué consecuencias, ventajas y limitaciones genera?

Comprender un patrón no significa memorizar su diagrama, sino reconocer el problema que resuelve y justificar cuándo conviene utilizarlo.

Introducción a los patrones creacionales

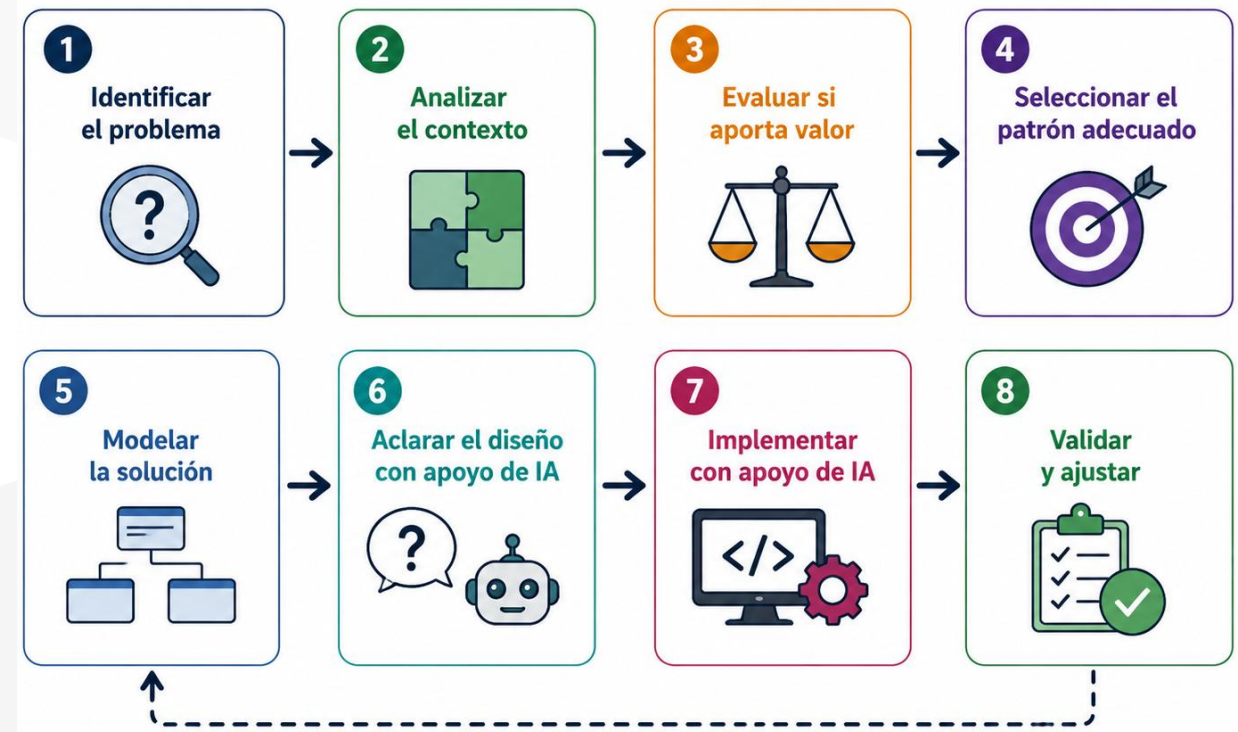
Plantilla de documentación de un patrón GoF

Campo	Descripción
Nombre y clasificación	Identifica el patrón y lo clasifica según su propósito y alcance.
Propósito	Resume qué hace el patrón, su finalidad y el problema de diseño que aborda.
También conocido como	Registra otros nombres utilizados para referirse al mismo patrón.
Motivación	Presenta un escenario que explica el problema y cómo la solución lo resuelve.
Aplicabilidad	Indica cuándo utilizar el patrón y cómo reconocer las situaciones apropiadas.
Estructura	Representa gráficamente las clases, los objetos y sus relaciones.
Participantes	Describe las clases u objetos involucrados y sus responsabilidades.
Colaboraciones	Explica cómo interactúan los participantes para cumplir sus responsabilidades.
Consecuencias	Expone resultados, beneficios, limitaciones y compromisos de la solución.
Implementación	Detalla técnicas, decisiones y dificultades relevantes para implementar el patrón.
Código de ejemplo	Ilustra una posible implementación en un lenguaje de programación.
Usos conocidos	Muestra aplicaciones del patrón identificadas en sistemas reales.
Patrones relacionados	Señala patrones semejantes, complementarios o utilizados como alternativas.

Introducción a los patrones creacionales

¿Cómo utilizar un patrón de diseño?

1. Identificar el problema: Reconocer el problema recurrente que se desea resolver.
2. Analizar el contexto: Comprender el dominio, las restricciones y los objetivos de la solución.
3. Evaluar si un patrón aporta valor: Verificar si el uso de un patrón mejora el diseño y si sus consecuencias son aceptables.
4. Seleccionar el patrón más adecuado: Elegir el patrón que mejor equilibra las fuerzas del problema.
5. Modelar la solución: Definir participantes, responsabilidades y colaboraciones mediante UML.
6. Aclarar el diseño con apoyo de IA: Entregar el modelo a la IA y solicitar preguntas que permitan aclarar supuestos, restricciones y decisiones aún no definidas.
7. Implementar con apoyo de IA: Generar código y pruebas a partir del UML y de las decisiones tomadas por el diseñador.
8. Validar y ajustar: Contrastar UML, código y pruebas; revisar críticamente la solución y corregir desviaciones.



Tema:

Factory Method

- Introducción a los patrones creacionales
- *Factory Method*
- Builder
- Singleton

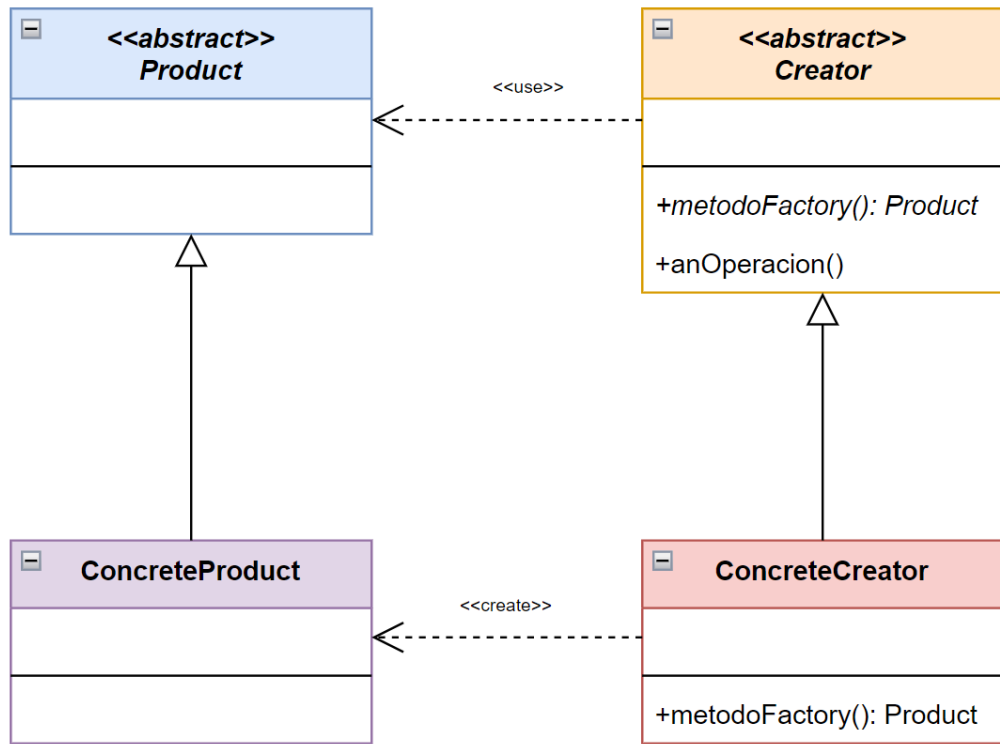
Factory Method

Factory

- Propósito
 - Define un método para crear objetos, permitiendo que las subclases decidan qué clase concreta instanciar.
 - Desacopla al código cliente de la creación directa de productos concretos.
- Problema / Motivación
 - Una empresa gestiona distintos tipos de contrato: Fijo, Temporal y por Factura.
 - Todos los contratos comparten una operación común, por ejemplo calcularSueldo(), pero cada tipo la implementa de manera diferente.
 - El sistema necesita crear contratos sin acoplar el código cliente directamente a ContratoFijo, ContratoTemporal o ContratoFactura.
 - La creación se delega a una jerarquía de creadores, donde cada subclase determina qué producto concreto debe instanciar.

Factory Method

Estructura



• Participantes

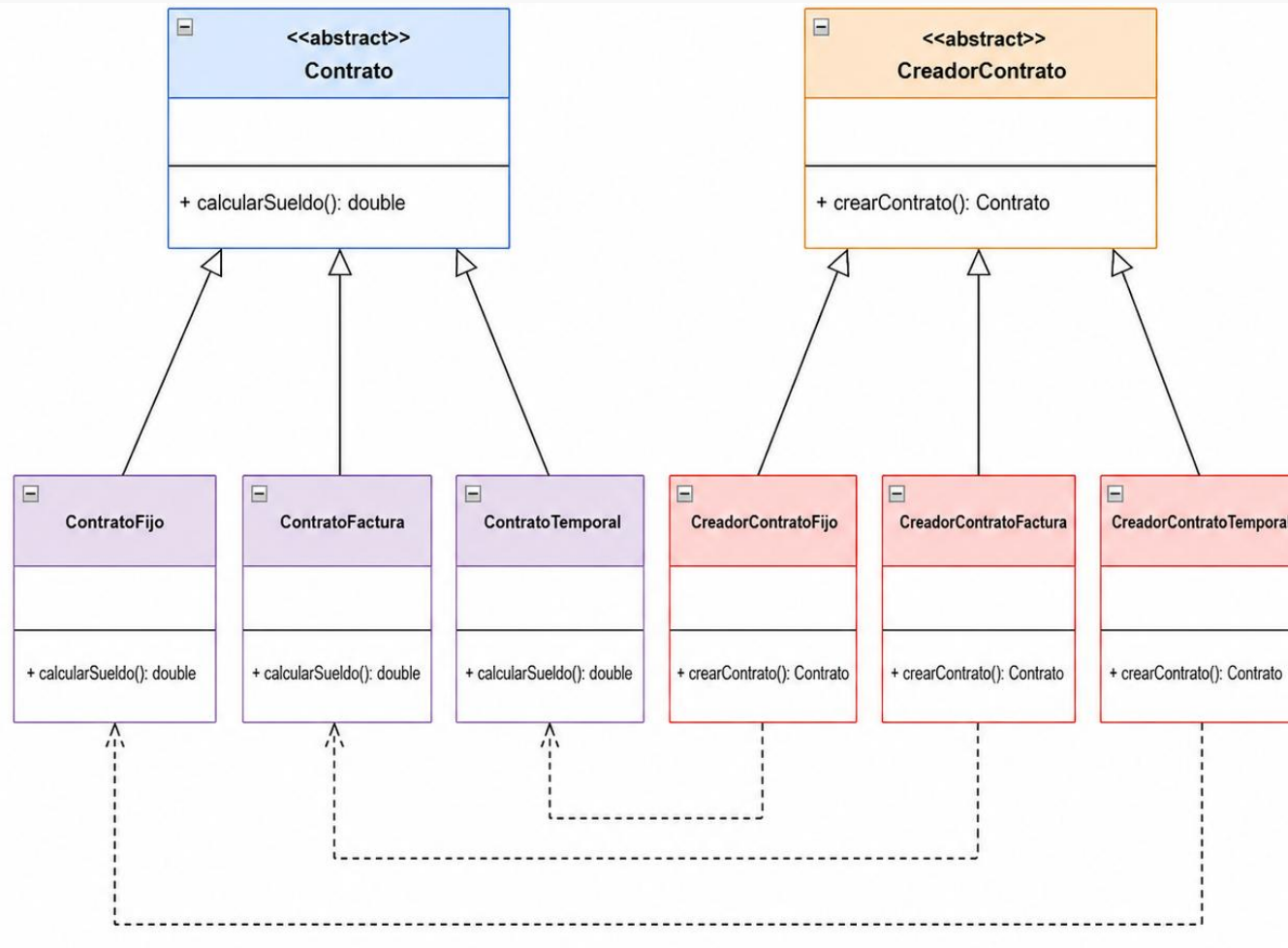
- **Product**: Define el contrato común de los objetos que puede crear el Factory Method. Puede representarse mediante una interfaz o una clase abstracta.
- **ConcreteProduct**: Implementa el contrato definido por Product.
- **Creator**: Declara el Factory Method, que devuelve un objeto de tipo Product.
- **ConcreteCreator**: Redefine el Factory Method para crear y devolver un ConcreteProduct.

• Colaboraciones

- Creator trabaja con la abstracción Product.
- Cada ConcreteCreator determina qué ConcreteProduct debe crear.
- El cliente puede utilizar el producto a través de su abstracción sin depender directamente de su clase concreta.

Factory Method

Solución



<https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/factoryMethod>

Factory Method

Factory

- Aplicabilidad
 - Una clase no puede anticipar qué clase concreta de objeto debe crear.
 - Se desea que las subclases decidan qué producto concreto instanciar.
 - Se quiere desacoplar el código cliente de las clases concretas que debe crear.
 - La creación de nuevos tipos de producto debe poder extenderse mediante nuevas subclases de Creator, evitando modificar el código cliente.

Factory Method resulta útil cuando la decisión de creación debe variar mediante herencia y polimorfismo, no mediante un switch centralizado.

Factory Method

Actividad en clase: ¿Aplicarían Factory Method?

Caso 1 — Logística y transporte

Una empresa de logística gestiona entregas nacionales e internacionales. El proceso de envío incluye registrar la carga, asignar un medio de transporte y realizar seguimiento. Las entregas terrestres utilizan camiones y las marítimas buques, pudiendo incorporarse nuevos medios en el futuro.

¿Cómo diseñarían la creación de los medios de transporte sin acoplar el proceso de envío a clases concretas?

Caso 2 — Gestión de documentos

Una plataforma empresarial procesa distintos tipos de documentos. Todos comparten operaciones como abrir, guardar y validar, pero cada módulo trabaja con un formato concreto: PDF, texto u hoja de cálculo. En el futuro podrían incorporarse nuevos formatos.

¿Cómo diseñarían la creación de documentos para incorporar nuevos formatos sin modificar continuamente el flujo de procesamiento?

Caso 3 — Videojuego y generación de enemigos

Un videojuego dispone de diferentes tipos de niveles. Todos siguen un proceso común de preparación, pero cada nivel requiere enemigos distintos; por ejemplo, un nivel medieval utiliza guerreros y uno futurista robots.

¿Cómo diseñarían la creación de enemigos para que cada nivel determine qué objetos concretos necesita?

Caso 4 — Generación de reportes

Un sistema institucional genera reportes académicos, financieros y administrativos. El proceso general consulta datos, los procesa y construye el reporte. Distintos módulos requieren formatos PDF, Excel o HTML, pudiendo aparecer nuevos formatos.

¿Cómo diseñarían la creación de reportes para evitar condicionales repetidos y facilitar nuevos formatos?

Tema:

Builder

- Introducción a los patrones creacionales
- Factory Method
- *Builder*
- Singleton

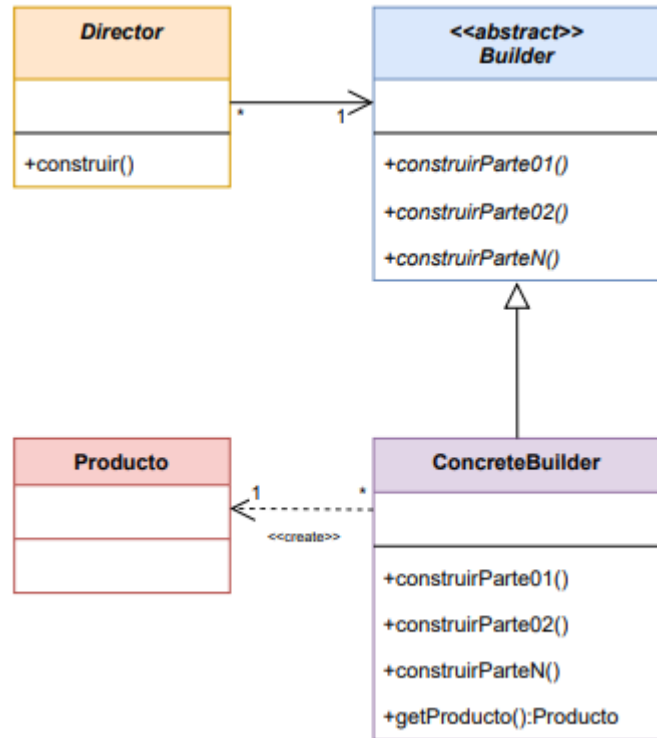
Builder

Builder

- Propósito
 - Separa la construcción de un objeto complejo de su representación, permitiendo que un mismo proceso de construcción produzca diferentes representaciones.
- Problema / Motivación
- Una empresa necesita construir objetos Renta para diferentes tipos de bienes raíces: departamentos, casas y terrenos.
 - Una renta puede incluir diferentes componentes, como: canon de arrendamiento; alícuota; agua; electricidad; internet; parqueadero; otros cargos.
 - Algunos componentes son obligatorios y otros opcionales según el tipo de propiedad.
 - La creación mediante constructores con numerosos parámetros dificultaría comprender qué información se está configurando y permitiría combinaciones poco claras.
 - Se necesita construir el objeto paso a paso, manteniendo un proceso común pero permitiendo diferentes configuraciones.

Builder

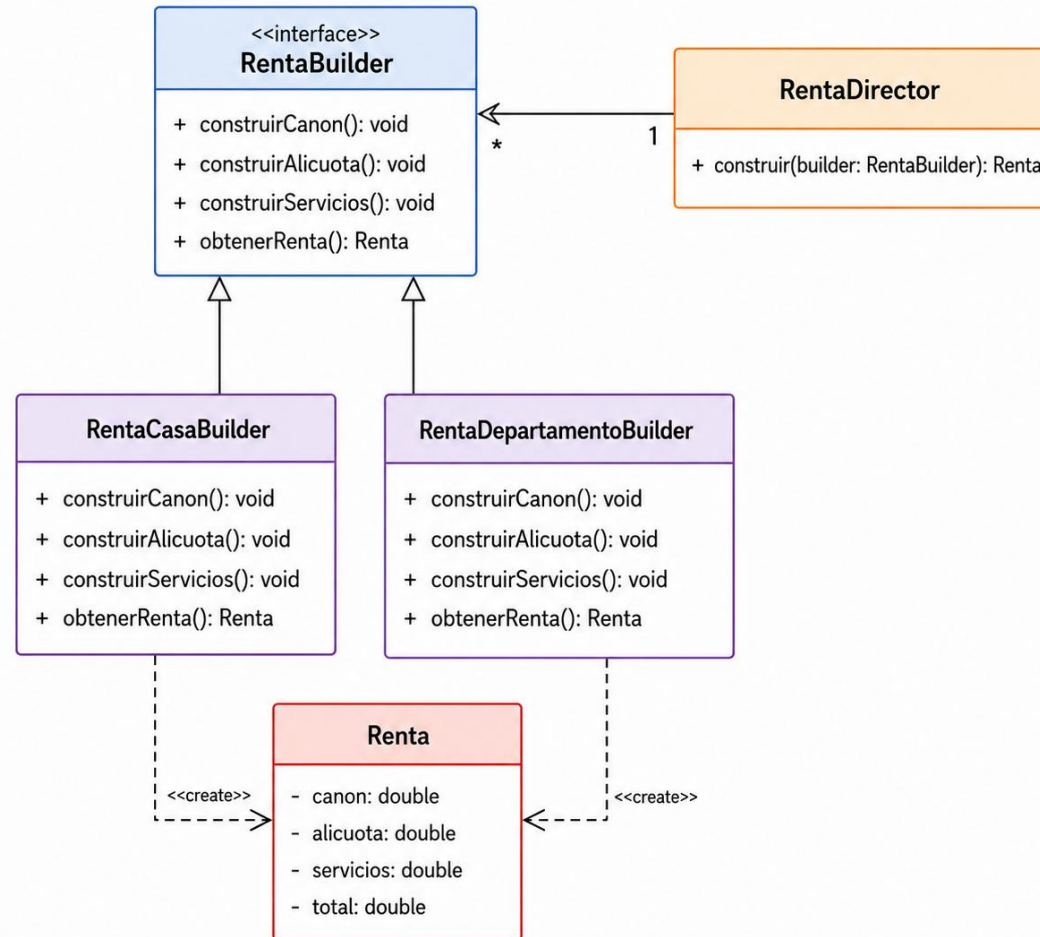
Estructura



- Participantes
 - **Builder**
Define las operaciones necesarias para construir las partes del producto.
 - **ConcreteBuilder:**
Implementa las operaciones de construcción. Mantiene la representación que se está construyendo. Permite obtener el producto final.
 - **Director:**
Coordina el proceso de construcción utilizando la interfaz Builder.
 - **Producto:**
Representa el objeto complejo resultante.
- Colaboraciones
 - El cliente selecciona un ConcreteBuilder.
 - El Director ejecuta la secuencia de construcción.
 - El ConcreteBuilder construye progresivamente el Product.
 - Al finalizar, el cliente obtiene el producto construido.

Builder

Solución



<https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/builder>

Builder

Builder

- Aplicabilidad
 - El proceso de construcción de un objeto complejo debe ser independiente de sus partes y de cómo se ensamblan.
 - Un mismo proceso de construcción debe permitir obtener diferentes representaciones del producto.
 - La construcción requiere varios pasos que conviene controlar de manera explícita.
 - Se desea separar la lógica de construcción del objeto resultante.

Builder es útil cuando el problema principal no es qué objeto crear, sino cómo construirlo progresivamente.

Variante moderna de Builder

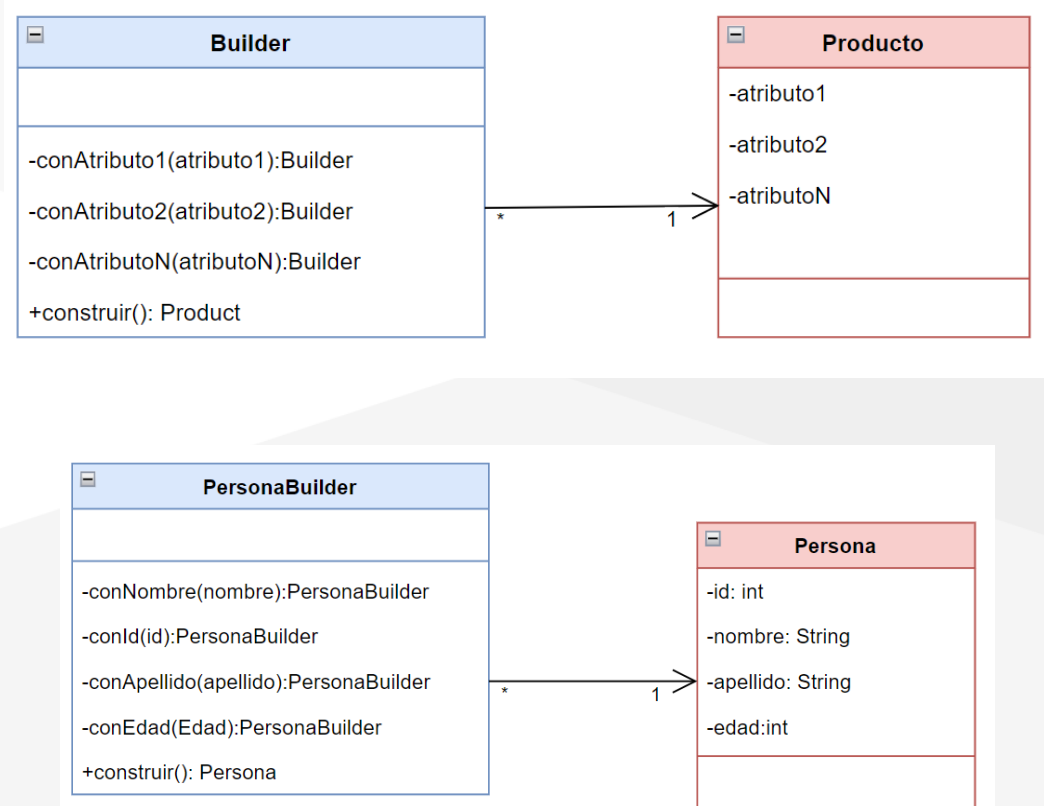
Solución

En implementaciones modernas, Builder suele utilizarse sin un objeto Director.

El cliente configura progresivamente el objeto mediante métodos encadenados y finalmente ejecuta build() para obtener el producto.

Resulta especialmente útil cuando:

- existen muchos parámetros;
- varios atributos son opcionales;
- se desea evitar múltiples constructores sobrecargados;
- se busca una construcción más legible y explícita.



<https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/mBuilder>

Builder

Actividad en clase: ¿Builder?

Caso 1 — Reserva de vuelo

Una reserva contiene origen, destino, fecha, pasajeros y tarifa. También puede incluir selección de asiento, equipaje adicional, alimentación y servicios especiales.

¿Cómo diseñarían la construcción progresiva de una reserva permitiendo configuraciones distintas y atributos opcionales?

Caso 2 — Personaje de videojuego

Un personaje posee nombre, nivel, raza, clase y habilidades. Además, puede incorporar armas, armaduras, objetos especiales y accesorios según su configuración.

¿Cómo diseñarían la construcción de personajes complejos sin utilizar constructores con demasiados parámetros?

Caso 3 — Reporte financiero

Un reporte puede incluir título, período, indicadores, tablas, gráficos, filtros, observaciones y diferentes secciones opcionales según las necesidades del usuario.

¿Cómo diseñarían la construcción del reporte para agregar sus componentes progresivamente?

Caso 4 — Menú de restaurante

Un menú puede componerse de entrada, plato principal, bebida, postre y complementos.

Algunas partes pueden ser opcionales y pueden existir diferentes configuraciones de menú.

¿Cómo diseñarían la construcción de diferentes menús utilizando un mismo proceso general y permitiendo variar sus componentes?.

Tema:

Singleton

- Introducción a los patrones creacionales
- Factory Method
- Builder
- *Singleton*

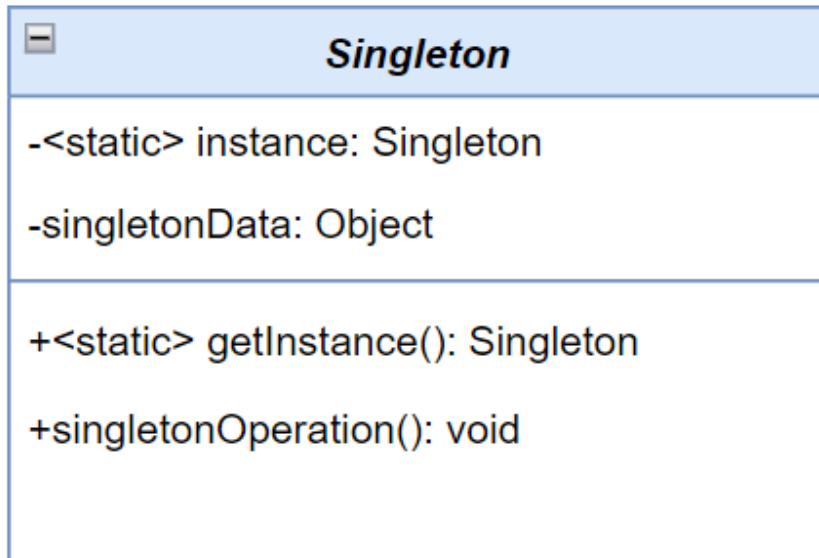
Singleton

Singleton

- Propósito
 - Garantiza que una clase tenga una única instancia y proporciona un punto de acceso controlado a ella.
- Problema / Motivación
 - Un sistema necesita compartir un recurso cuya existencia debe estar controlada durante la ejecución de la aplicación.
 - Por ejemplo, una aplicación utiliza un componente de configuración global que contiene parámetros comunes para distintos módulos.
 - Si cada módulo crea su propia instancia, podrían existir configuraciones inconsistentes o duplicadas.
 - Se necesita:
 - controlar la creación de la instancia;
 - garantizar que exista una sola instancia dentro del alcance definido;
 - proporcionar un mecanismo conocido para acceder a ella.

Singleton

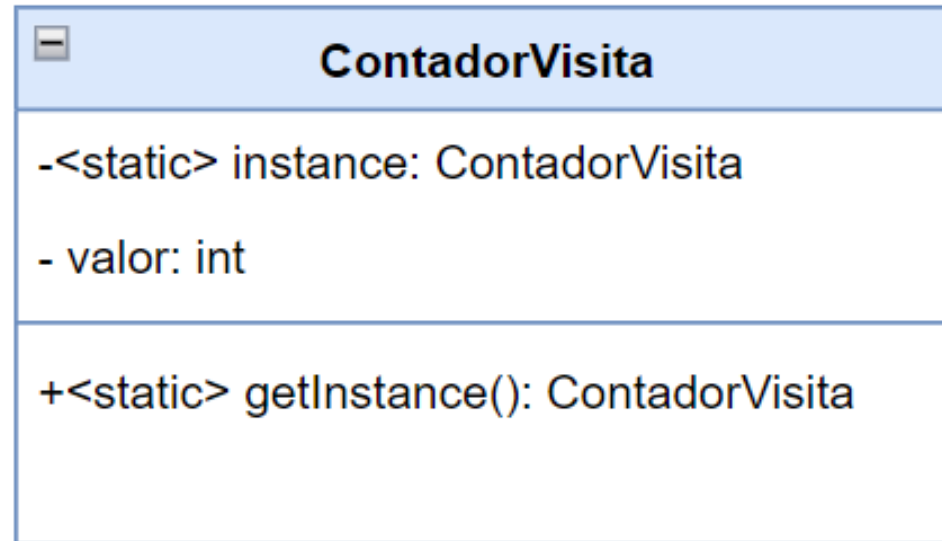
Estructura



- Participantes
 - Singleton
 - Mantiene una referencia a su única instancia.
 - Controla su propia creación.
 - Proporciona una operación pública para acceder a la instancia.
 - Evita que los clientes creen objetos directamente mediante un constructor no público.
- Colaboraciones
 - Los clientes solicitan la instancia mediante una operación como getInstance().
 - La clase crea la instancia cuando corresponde y devuelve siempre la misma referencia.
 - Los clientes utilizan la instancia sin conocer ni controlar directamente su creación..

Singleton

Solución



<https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/singleton>

Singleton

Singleton

- Aplicabilidad
 - Deba existir una única instancia controlada de una clase dentro de un alcance determinado.
 - Esa instancia represente un recurso o servicio compartido por varios componentes.
 - Sea necesario controlar explícitamente cómo se crea y cómo se accede a la instancia.
 - La existencia de múltiples instancias pueda provocar inconsistencias o conflictos.
- Evitar utilizarlo cuando:
 - Solo se busca disponer de una variable global.
 - La clase tiene demasiadas responsabilidades.
 - Dificulta las pruebas unitarias o introduce dependencias ocultas.
 - El mismo problema puede resolverse mejor mediante inyección de dependencias.

Singleton

Actividad en clase: ¿Singleton?

Caso 1 — Configuración de aplicación

Una aplicación de escritorio necesita cargar parámetros comunes desde un archivo de configuración. Diferentes módulos requieren consultar esos valores durante la ejecución.

¿Justificarían una única instancia de configuración? ¿Qué problema evitaría Singleton?

Caso 2 — Gestor de impresión

Una aplicación controla una impresora local y coordina los trabajos enviados desde distintos módulos.

¿Conviene que exista una única instancia del gestor?
¿Qué ocurriría si existieran varias?

Caso 3 — Sesión de usuario en una aplicación web

El sistema maneja múltiples usuarios autenticados simultáneamente.

¿Aplicarían Singleton para representar la sesión?
Justifiquen su respuesta.

Caso 4 — Acceso a base de datos

Una aplicación necesita realizar muchas operaciones concurrentes contra una base de datos.

¿Utilizarían Singleton para mantener una única conexión? ¿Qué alternativa sería más adecuada?.

Unidad 01.03

Retroalimentación y cierre

Qué vimos hoy

- Factory Method: Decide qué objeto concreto crear mediante herencia y polimorfismo.
- Builder: Organiza cómo construir progresivamente un objeto complejo.
- Singleton: Controla cuántas instancias pueden existir dentro de un alcance determinado.

Antes de finalizar:

- ¿Qué responsabilidad de creación asume cada patrón?
- ¿Qué problema resuelve mejor cada uno?
- ¿Qué riesgo aparece si se utiliza el patrón sin necesidad?

Idea final: Los patrones creacionales no eliminan la creación de objetos; asignan mejor la responsabilidad de decidir cómo, qué y cuántos objetos crear.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Anexos

Links de Referencia

- <https://www.campusmvp.es/recursos/post/los-conceptos-fundamentales-sobre-programacion-orientada-objetos-explicados-de-manera-simple.aspx>
- <https://codideep.com/blogpost/stupid-vs-solid>
- <https://refactoring.guru/design-patterns>
- <https://reactiveprogramming.io/books/design-patterns/es>

Gracias por su atención

posgrados.ups.edu.ec